

SINGER — Training Pipeline (V9)

Behavioural Cloning + DAgger with CLIPSeg Centroid Features

Contents

1	Notation	2
2	Architecture — the policy π_θ	2
3	Centroid features c^k — the V9 addition	2
4	Stage 1 — Behavioural Cloning (BC)	3
4.1	Data collection	3
4.2	Loss — pure MSE on the action	3
5	Stage 2 — DAgger V9	3
5.1	Mixed-policy rollout	3
5.2	Expert-mixing schedule	4
5.3	Annotation aggregation	4
5.4	Retraining loss — BC, restricted to two heads	4
5.5	Best-checkpoint reset	4
5.6	Annotation filters	5
6	One DAgger iteration — algorithmic summary	5
7	Hyperparameters — V9 reference run	5
8	Final benchmark — V9 canonical results	6

1 Notation

Symbol	Dim	Meaning
k	$\mathbb{N}$	control timestep ($\Delta t = 50$ ms, 20 Hz)
I^k	$H \times W \times 3$	RGB camera image at step k
$v_{\mathcal{W}}^k$	$\mathbb{R}^3$	linear velocity in world frame
$q_{\mathcal{BW}}^k$	$\mathbb{R}^4$	body→world quaternion
$p_{z\mathcal{W}}^k$	$\mathbb{R}$	altitude (z-position)
u^k	$\mathbb{R}^4$	action: $[k_{th}, \omega_x, \omega_y, \omega_z]$
$\delta^{k-5:k-1}$	—	5-step history window of state-deltas + actions
o^k	$\mathbb{R}^d$	flattened observation passed to network
h^k	$\mathbb{R}^8$	history embedding (HistoryMLP output)
ϕ^k	$\mathbb{R}^{256}$	visual embedding (SqueezeNet + FE MLP output)
c^k	$\mathbb{R}^3$	centroid features (V9) : bearing, elevation, app. size
π_θ	—	policy network (Commander head)
β	$[0, 1]$	DAgger expert-mixing coefficient
$\mathcal{D}_{\text{BC}}$	158 K	behavioural cloning dataset (single scene)
$\mathcal{D}_{\text{DAg}}$	growing	aggregated DAgger annotations

2 Architecture — the policy π_θ

The visuomotor policy is a multi-branch network that fuses **visual**, **proprioceptive**, **historical** and (V9) **centroid** features into a single 4-D rate command.

$$\phi^k = \text{FE-MLP}(\text{SqueezeNet}(\text{CLIPSeg}(I^k))) \quad (\text{visual}) \quad (1)$$

$$c^k = \Phi_{\text{centroid}}(\text{CLIPSeg}(I^k)) \quad (\text{V9 NEW}) \quad (2)$$

$$h^k = \text{HistoryMLP}(\delta t^{k-5:k-1}, \delta p_{\mathcal{W}}^{k-5:k-1}, \delta v_{\mathcal{W}}^{k-5:k-1}, \delta q_{\mathcal{BW}}^{k-5:k-1}, u^{k-5:k-1}) \quad (\text{temporal}) \quad (3)$$

$$o^k = [\phi^k \parallel c^k \parallel v_{\mathcal{W}}^k \parallel q_{\mathcal{BW}}^k \parallel p_{z\mathcal{W}}^k \parallel h^k] \quad (\text{concat}) \quad (4)$$

$$\hat{u}^k = \pi_\theta(o^k) = \text{ActionHead}_{[100,100]}(o^k) \quad (\text{action}) \quad (5)$$

The CLIPSeg backbone is *frozen* (zero-shot, prompt = object name). Only the History MLP, Feature Extractor MLP and Action Head are learned. The centroid extractor Φ_{centroid} is purely analytical (no parameters).

3 Centroid features c^k — the V9 addition

Let $\mathbf{H} \in [0, 1]^{H \times W}$ be the CLIPSeg sigmoid heatmap for the text prompt (*e.g.* “green clock”). Define the bright-pixel set

$$M = \{ (x, y) \mid \mathbf{H}(x, y) > P_{75}(\mathbf{H}) \}.$$

$$c_x = \frac{\sum_{(x,y) \in M} x \mathbf{H}(x,y)}{\sum_{(x,y) \in M} \mathbf{H}(x,y)}, \quad c_y = \frac{\sum_{(x,y) \in M} y \mathbf{H}(x,y)}{\sum_{(x,y) \in M} \mathbf{H}(x,y)} \quad (6)$$

$$\text{bearing} = 2 c_x / W - 1, \quad \text{elevation} = 2 c_y / H - 1, \quad \text{app. size} = |M| / (H \cdot W) \quad (7)$$

$$c^k = [\text{bearing}, \text{elevation}, \text{app. size}]^\top \quad (8)$$

Intuition. CLIPSeg gives a 352×352 heatmap; instead of feeding that giant tensor to the policy, we collapse it into 3 scalars that encode *where* the target is (bearing, elevation) and *how big* it appears (distance proxy). They are concatenated into the state vector — no new trainable parameters.

Known V9 caveat. The same P_{75} threshold is used for both the centroid mask and the size mask, so $|M|/(HW) \approx 0.25$ is structurally constant. The +44 pp BC lift is therefore driven by **bearing** + **elevation** alone. V10 fixes this with a fixed sigmoid threshold for size.

4 Stage 1 — Behavioural Cloning (BC)

4.1 Data collection

A sampling-based motion planner (RRT* with semantic goal sets) generates expert trajectories from random start poses to each target object in the flightroom Gaussian-Splat scene.

Quantity	Value	Note
trajectory length	6 s	at 20 Hz $\rightarrow$ 120 steps
trajectories / object	110	3 objects: clock, leafblower, drill-on-boxes
augmentations / traj	4	domain-random visuals
total samples	158,400	$= 110 \times 3 \times 4 \times 120$
control rate	20 Hz	$\Delta t = 50$ ms

4.2 Loss — pure MSE on the action

$$\mathcal{L}_{\text{BC}}(\theta) = \mathbb{E}_{(o^k, u_\star^k) \sim \mathcal{D}_{\text{BC}}} \left[\|\pi_\theta(o^k) - u_\star^k\|_2^2 \right] \quad (9)$$

where $u_\star^k$ is the expert (RRT-MPC) action at step k . The loss is implemented as `nn.MSELoss(reduction='mean')` in `src/sousvide/instruct/train_policy.py`, optimised with Adam at $\eta = 10^{-4}$, batch size 64, for $N_{\text{ep}}^{\text{com}} = 150$ epochs on the Commander head and $N_{\text{ep}}^{\text{his}} = 100$ on the History MLP. Best-checkpoint selection is done on a held-out validation split (every 5 epochs).

5 Stage 2 — DAgger V9

DAgger (Dataset Aggregation, Ross et al., 2011) closes the BC distribution gap by rolling out a *mixed* expert/learner policy in the simulator, then retraining the learner on the actions the expert would have taken at the states the learner actually visited.

5.1 Mixed-policy rollout

At each step k , both the MPC expert and the BC pilot are queried:

$$u_{\text{exp}}^k = \text{MPC}(x^k, o^k) \quad (\text{expert command}) \quad (10)$$

$$u_{\text{pil}}^k = \pi_{\theta}(o^k) \quad (\text{pilot command}) \quad (11)$$

$$u_{\text{exec}}^k = \begin{cases} u_{\text{exp}}^k & \text{w.p. } \beta_t \\ u_{\text{pil}}^k & \text{w.p. } 1 - \beta_t \end{cases} \quad (\text{executed in sim}) \quad (12)$$

$$\mathcal{D}_{\text{DAg}} \leftarrow \mathcal{D}_{\text{DAg}} \cup \{(o^k, u_{\text{exp}}^k)\}_k \quad (\text{annotation}) \quad (13)$$

Crucially, the *annotation label is always* u_{exp}^k , even when the executed action was the pilot's. This is what gives DAgger its no-regret guarantee. Implementation: `class MixedPolicy in train_dagger.py:951`.

5.2 Expert-mixing schedule

β decays geometrically across iterations so the rollouts become progressively pilot-driven:

$$\beta_t = \beta_{\text{start}} \cdot \beta_{\text{decay}}^t \quad \text{with} \quad \beta_{\text{start}} = 0.80, \beta_{\text{decay}} = 0.92 \quad (14)$$

For $T = 12$ iterations: $\beta_0 = 0.80, \beta_1 = 0.736, \dots, \beta_{11} \approx 0.32$.

5.3 Annotation aggregation

V9 uses **cumulative** aggregation (`aggregate_dagger: true`):

$$\mathcal{D}_{\text{DAg}}^{(t)} = \mathcal{D}_{\text{DAg}}^{(t-1)} \cup \mathcal{D}_{\text{new}}^{(t)} \quad (\text{classic DAgger}) \quad (15)$$

Annotations are also **oversampled** ($\times 20$ duplication, `dagger_oversample: 20`) before being mixed with the BC dataset to counteract their numerical minority.

5.4 Retraining loss — BC, restricted to two heads

After each iteration the policy is fine-tuned on the $(\text{BC} \cup \text{DAgger})$ mixture. The objective is identical in form to Eq. (9), but the **VisionMLP is frozen** (only the CommanderSV head and the small MLP on top of the centroid + state stack are updated):

$$\theta = \theta_{\text{CommanderSV}} \quad (\text{VisionMLP frozen — prevents catastrophic forgetting}) \quad (16)$$

$$\mathcal{L}_{\text{DAg}}(\theta) = \mathbb{E}_{(o^k, u_{\text{exp}}^k) \sim \mathcal{D}_{\text{BC}} \cup \mathcal{D}_{\text{DAg}}} \left[\|\pi_{\theta}(o^k) - u_{\text{exp}}^k\|_2^2 \right] \quad (17)$$

Adam at $\eta_{\text{DAg}} = 2 \times 10^{-5}$ ($5 \times$ smaller than BC), $N_{\text{ep}}^{\text{DAg}} = 10$ epochs per iteration, batch size 64. Implementation: `_retrain_commander` in `train_dagger.py:2028`.

5.5 Best-checkpoint reset

`reset_to_best: true` — if validation success rate at iteration t is worse than the best so far, the model weights are restored from the best checkpoint before iteration $t + 1$. Patience = 5 iterations before early stopping. This is what stopped the runaway-regression failure mode that plagued earlier DAgger versions.

5.6 Annotation filters

To prevent garbage-in-garbage-out, an annotation (o^k, u_{exp}^k) is *rejected* if any of the following hold:

Filter	Threshold	Purpose
position deviation	$> 8 \text{ m}$	runaway pilot, expert query meaningless
goal distance	$> 50 \text{ m}$	lost-in-space trajectories
orientation deviation	$> 90^\circ$	pilot pointing wrong way
close-approach distance	$< 5 \text{ m}$	near-goal hover regime, low signal

6 One DAgger iteration — algorithmic summary

Step	Operation	Notes
1	Query mixed policy in sim ($n_{\text{rol}}=8/\text{object}$, 3 objects)	generates ≈ 3000 steps
2	Filter annotations (deviation, goal-dist, orient.)	drop $\sim 30\text{--}50\%$ of raw
3	Aggregate: $\mathcal{D}_{\text{DAg}} \leftarrow \mathcal{D}_{\text{DAg}} \cup \mathcal{D}_{\text{new}}$	cumulative
4	Oversample $\mathcal{D}_{\text{DAg}}$ by $\times 20$	balance vs BC data
5	Mix with $\mathcal{D}_{\text{BC}}$ (158 K)	symlink BC observation dirs
6	Retrain Commander only ($\eta = 2 \times 10^{-5}$, 10 ep)	VisionMLP frozen
7	Evaluate on 40 held-out trajectories	success / collision / goal-dist
8	If worse than best $\rightarrow$ reset to best checkpoint	else update best
9	$\beta \leftarrow \beta \cdot 0.92$	geometric decay

7 Hyperparameters — V9 reference run

Symbol	Value	Description
<i>BC stage — ssv_BC_CENTROID_V9</i>		
$N_{\text{ep}}^{\text{com}}$	150	Commander epochs
$N_{\text{ep}}^{\text{his}}$	100	History MLP epochs
η_{BC}	10^{-4}	Adam learning rate
batch size	64	—
<i>DAgger stage — SSV_DAGGER_CENTROID_V9</i>		
T	12	DAgger iterations
β_{start}	0.80	initial expert mix
β_{decay}	0.92	geometric decay/iter
$N_{\text{ep}}^{\text{DAg}}$	10	retraining epochs/iter
η_{DAg}	2×10^{-5}	retraining LR
oversample	$20\times$	DAgger annotation duplication
$n_{\text{rol}}/\text{obj}$	8	rollouts per object per iter
n_{eval}	40	per-iter eval trajectories
aggregate_dagger	true	cumulative annotation set
reset_to_best	true	rollback on regression
freeze_vision	true	VisionMLP frozen during retrain
patience	5 iters	early stop
seed	42	deterministic eval

8 Final benchmark — V9 canonical results

Metric	Pre-Centroid BC	Dagger V7	Centroid BC (V9)	Centroid DAgger (V9)
Success	36.7%	52.3%	80.7%	88.0%
Collision	21.3%	$\sim 20\%$	13.3%	8.0%
Goal dist (m)	3.71	~ 2.5	1.84	1.65
Clock	—	45%	86%	90%
Leafblower	—	60%	76%	84%
Drill on boxes	—	55%	80%	90%

Headline lifts.

- Centroid features alone: 36.7% $\rightarrow$ 80.7% (+44.0 pp, BC vs BC)
- DAgger on top of centroid: 80.7% $\rightarrow$ 88.0% (+7.3 pp)
- End-to-end (Pre-Centroid BC $\rightarrow$ V9 DAgger): +51.3 pp success, -13.3 pp collision, goal-dist 3.71 $\rightarrow$ 1.65 m

Take-away. Most of the gain comes from the *features*, not from DAgger. Collapsing the CLIPSeg heatmap into 3 scalars gives the BC policy enough inductive bias that DAgger has comparatively little left to fix — but the residual lift it does provide concentrates on the hardest object (drill-on-boxes, +10 pp) and uniformly reduces collisions across the board.